Sentencias DML / DDL

1.1 Introducción

Una vez comprendidos los fundamentos y la arquitectura general de **JDBC**, el siguiente paso consiste en **aprender a ejecutar sentencias SQL desde una aplicación Java**.
Estas sentencias permiten **crear, modificar, eliminar o consultar datos** almacenados en una base de datos relacional, convirtiéndose en el núcleo del acceso a datos mediante JDBC.

El **mecanismo de ejecución** se basa en la creación de **objetos específicos** que actúan como intermediarios entre la aplicación Java y el gestor de base de datos. A través de estos objetos se envían órdenes SQL, se reciben los resultados y se controlan los posibles errores o excepciones.

En JDBC existen **tres tipos principales de sentencias**, cada uno con un propósito distinto:

- **Statement**, para ejecutar instrucciones SQL **fijas** (sin parámetros).
- **PreparedStatement**, para ejecutar **consultas parametrizadas**, seguras y reutilizables.
- **CallableStatement**, para **invocar procedimientos almacenados** definidos en la base de datos.

El dominio de estas tres clases permite construir aplicaciones robustas, seguras y eficientes, capaces de manipular los datos directamente desde Java.
A lo largo de este apartado se estudiará el uso, las características y los ejemplos prácticos de cada uno de ellos, siguiendo un enfoque progresivo desde las sentencias más simples (Statement) hasta las más complejas (CallableStatement).

1.2 Tipos de sentencias

En JDBC existen tres tipos fundamentales de objetos que permiten ejecutar instrucciones SQL contra una base de datos.
Estos objetos se diferencian por su complejidad, el tipo de sentencias que aceptan y su nivel de seguridad frente a posibles errores o ataques como la **inyección SQL**.

```
java.sql.Statement
    ├── PreparedStatement
    └── CallableStatement
```

Todos los objetos derivan de la clase **Statement**, que representa la forma básica de ejecutar SQL.

PreparedStatement y **CallableStatement** heredan de ella y añaden nuevas funcionalidades especializadas.

Clase	Uso principal	Admite parámetros	Propósito típico	Ejemplo de ejecución
Statement	Ejecutar SQL estático sin variables	No	Crear tablas, borrar registros fijos, realizar consultas simples	stmt.execute("CREATE TABLE...")
PreparedStatement	Ejecutar SQL precompilado con variables	Sí (?)	Insertar, actualizar o eliminar registros según datos del usuario	pst.executeUpdate()
CallableStatement	Ejecutar procedimientos o funciones almacenadas	IN / OUT / INOUT	Llamar a procedimientos definidos en la BD	cs.execute()

Funcionamiento general

1. Se establece la conexión con la base de datos (Connection).
2. A partir de esa conexión se crea el objeto adecuado:

```
Statement stmt = con.createStatement();
PreparedStatement pst = con.prepareStatement(sql);
CallableStatement cs = con.prepareCall("{ call procedimiento(?) }");
```

3. Se ejecuta la sentencia SQL mediante uno de los siguientes métodos:
   - execute() → ejecuta cualquier tipo de sentencia (DDL, DML o consulta).
   - executeQuery() → ejecuta una consulta SELECT y devuelve un ResultSet.
   - executeUpdate() → ejecuta INSERT, UPDATE o DELETE y devuelve el número de filas afectadas.
4. Finalmente, se liberan los recursos (ResultSet, Statement, Connection) — preferiblemente mediante **try-with-resources**.

1.3 Clase Statement

Concepto general

La clase **Statement** es el objeto más sencillo y directo de JDBC para ejecutar sentencias SQL.
Permite enviar al gestor de base de datos **órdenes SQL fijas** (sin variables ni parámetros) que pueden ser de distintos tipos:

- **DDL (Data Definition Language):** crear, modificar o eliminar tablas y estructuras (CREATE, ALTER, DROP).
- **DML (Data Manipulation Language):** insertar, actualizar o eliminar registros (INSERT, UPDATE, DELETE).
- **SELECT:** consultas de datos sin parámetros dinámicos.

Aclaración: una **sentencia SQL sin parámetros** es aquella que **no contiene valores variables**, sino que el SQL se construye completamente antes de su ejecución. Ejemplo:

```
String sql = "SELECT * FROM EMPLEADOS WHERE DEP_NO = 30";
```

Aquí el valor “30” está escrito directamente en el código, no se reemplaza dinámicamente.

```
String sql = "SELECT * FROM EMPLEADOS WHERE APELLIDO = '" + nombre + "'";
```

En este caso, el usuario podría escribir código malicioso (inyección SQL).
Por eso en estas situaciones se recomienda PreparedStatement.

Cuando usarlo:

- Para comandos **muy simples o puntuales** (por ejemplo, crear una tabla o eliminar todos los registros).
- Cuando **no hay variables** en la sentencia SQL.

Inconvenientes:

- Poco eficiente si se ejecuta repetidamente.
- Vulnerable a **inyección SQL** si se construyen consultas concatenando variables del usuario

Sintaxis

```
Statement stmt = con.createStatement();
```

A través de este objeto se pueden ejecutar **tres tipos de métodos**:

Método	**Uso principal**	**Devuelve**
execute(sql)	Ejecuta cualquier instrucción SQL	true si devuelve un ResultSet, false si no
executeQuery(sql)	Ejecuta consultas SELECT	Un objeto ResultSet
executeUpdate(sql)	Ejecuta INSERT, UPDATE o DELETE	Número de filas afectadas

IMPORTANTE: Revisar los ejemplos del aula virtual (Statement.zip) que contiene un proyecto Maven con ejemplos de uso de Statement.

1. 4 Clase PreparedStatement

Concepto general

La clase **PreparedStatement** es una extensión de Statement que permite **definir consultas SQL con parámetros**, representados por el signo de interrogación ?. Estos parámetros se sustituyen posteriormente por valores concretos mediante métodos como setString(), setInt(), setDouble(), etc.

El motor JDBC **precompila** la sentencia SQL en el servidor de base de datos, lo que aporta dos ventajas clave:

1. **Mayor seguridad**, al separar la estructura de la consulta de los datos, evitando ataques de inyección SQL.
2. **Mejor rendimiento**, porque la sentencia se analiza y optimiza una sola vez y puede reutilizarse varias veces con distintos valores.

Ventajas principales

- **Evita la inyección SQL**, ya que los parámetros no se concatenan en la cadena SQL.
- **Mejora el rendimiento**, porque el SGBD **precompila** la sentencia y puede reutilizarla.
- **Simplifica el código** en operaciones repetitivas (por ejemplo, insertar muchos registros).

Métodos principales

Método	Descripción
setInt(posición, valor)	Asigna un número entero al parámetro indicado
setString(posición, valor)	Asigna una cadena de texto
setDouble(posición, valor)	Asigna un número decimal
setDate(posición, valor)	Asigna una fecha SQL (java.sql.Date)
executeQuery()	Ejecuta una sentencia SELECT y devuelve un ResultSet
executeUpdate()	Ejecuta INSERT, UPDATE o DELETE
clearParameters()	Limpia los valores de los parámetros antes de reutilizar la consulta

IMPORTANTE: Revisar los ejemplos del aula virtual (PreparedStatement.zip) que contiene un proyecto Maven con ejemplos de uso de Statement.

1. 5 Clase CallableStatement

Concepto general

La clase **CallableStatement** es una extensión de PreparedStatement que permite **ejecutar procedimientos y funciones almacenadas** directamente en el servidor de base de datos.

Un **procedimiento almacenado (Stored Procedure)** es un bloque de instrucciones SQL que se guarda en el servidor y se ejecuta bajo un nombre determinado. Estos procedimientos pueden recibir **parámetros de entrada (IN)**, **de salida (OUT)** o **mixtos (INOUT)**.

El uso de CallableStatement en JDBC facilita:

- La reutilización de lógica compleja del lado del servidor.
- Una comunicación más eficiente con la base de datos.
- Un mejor control de la integridad y la seguridad de las operaciones.

Sintaxis

```
CallableStatement cs = con.prepareCall("{CALL nombreProcedimiento(?, ?,
?)}");
```

Cómo se usa

El flujo general es:

```
// 1. Definir la llamada SQL con la sintaxis del procedimiento o función
String sql = "{CALL nombreProcedimiento(?, ?, ?)}"; // Procedimiento
// o bien
String sql = "{? = CALL nombreFuncion(?)}";         // Función

// 2. Crear la conexión y el CallableStatement
Connection con = ConexionBD.conectar();
CallableStatement cs = con.prepareCall(sql);

// 3. Registrar y asignar parámetros (si los hay)
cs.setInt(1, 10);  // Parámetro IN
cs.registerOutParameter(2, Types.DOUBLE); // Parámetro OUT

// 4. Ejecutar
cs.execute();

// 5. Recuperar valores devueltos (si los hay)
double resultado = cs.getDouble(2);
```

Tipos y orden de parámetros

Procedimientos

Los tipos de parámetros son:

Tipo	Significado	Ejemplo en SQL	Ejemplo en Java
IN	El programa envía un valor al procedimiento.	IN p_id INT	cs.setInt(1, 10);
OUT	El procedimiento devuelve un valor a Java.	OUT total INT	cs.registerOutParameter(1, Types.INTEGER); → cs.getInt(1);
INOUT	Se envía un valor inicial y el procedimiento puede modificarlo y devolverlo.	INOUT valor DECIMAL(10,2)	cs.setDouble(1, 5.5); + cs.registerOutParameter(1, Types.DOUBLE);

Cada **parámetro** (ya sea de entrada, salida o mixto) tiene una **posición numérica** dentro de la llamada SQL, y esa posición se indica **con un índice empezando por 1**.

Cuando defines un procedimiento o función en SQL, los parámetros aparecen **en un orden concreto**, por ejemplo:

```
CREATE PROCEDURE ejemplo(
    IN p_id INT,
    IN p_nombre VARCHAR(50),
    OUT p_resultado INT
)
```

El orden sería:

Posición	Nombre SQL	Tipo	En Java se referencia como...
1	p_id	IN	cs.setInt(1, 123)
2	p_nombre	IN	cs.setString(2, "Ana")
3	p_resultado	OUT	cs.registerOutParameter(3, Types.INTEGER)

Cómo se asignan en Java:

```
String sql = "{CALL ejemplo(?, ?, ?)}";
CallableStatement cs = con.prepareCall(sql);
```

- Cada **?** representa un parámetro.
- La posición del primer ? es **1**, la del segundo **2**, y así sucesivamente.

Funciones

En las **funciones**, la posición 1 siempre corresponde al **valor devuelto por la función**, y los demás parámetros empiezan a contarse desde **2**.

Ejemplo:

```
CREATE FUNCTION calcularSalarioAnual(p_salario DECIMAL(10,2))
RETURNS DECIMAL(10,2)
BEGIN
  RETURN p_salario * 12;
END;
```

En Java:

```
String sql = "{? = CALL calcularSalarioAnual(?)}";
CallableStatement cs = con.prepareCall(sql);

// 1 → valor devuelto (RETURN)
cs.registerOutParameter(1, Types.DECIMAL);

// 2 → parámetro de entrada p_salario
cs.setDouble(2, 1500.00);

cs.execute();
System.out.println("Salario anual: " + cs.getDouble(1));
```

Resumen:

Tipo de llamada	Ejemplo SQL	Ejemplo Java	Posición de los parámetros
Procedimiento con parámetros IN	CALL aumentarSalario(30, 10.0)	cs.setInt(1, 30);cs.setDouble(2, 10.0);	1, 2, ... (uno por cada ?)
Procedimiento con parámetro OUT	CALL contarEmpleados(@total)	cs.registerOutParameter(1, Types.INTEGER);cs.execute();int total = cs.getInt(1);	1, 2, ... (según orden en la definición SQL)
Procedimiento con parámetro INOUT	CALL doblarNumero(@valor)	cs.setInt(1, 5);cs.registerOutParameter(1, Types.INTEGER);cs.execute();int resultado = cs.getInt(1);	Mismo índice para entrada y salida
Función con valor de retorno	SELECT calcularSalarioAnual(1500);	String sql = "{? = CALL calcularSalarioAnual(?)}";cs.registerOutParameter(1, Types.DECIMAL);cs.setDouble(2, 1500.00);cs.execute();double salarioAnual = cs.getDouble(1);	1 es el valor devuelto (RETURN)2, 3... son los parámetros de entrada

IMPORTANTE: Revisar los ejemplos del aula virtual (CallableStatement.zip) que contiene un proyecto Maven con ejemplos de uso de Statement.